MÔN NHẬN DẠNG MẪU (PATTERN RECOGNITION)

Nhận dạng chữ số viết tay MNIST

Bốn bộ phân lớp tự cài đặt (Gaussian Naïve Bayes, hồi quy softmax, MLP, AdaBoost-SAMME) và một CNN nhỏ

Đồ án giữa kỳ

So sánh trên cùng một giao thức không rò rỉ dữ liệu, có đối chiếu với scikit-learn; mở rộng sang Fashion-MNIST, EMNIST Digits, tăng cường dữ liệu và trình khám phá thực thi chạy trong trình duyệt.

Thành viên	MSSV	Đóng góp
Trần Trinh Nhân	24280088	50%
Lâm Nhật Tiến	24280029	50%

Mã nguồn: `https://github.com/TranTrinhNhan1/WriteRHand`

Ngày biên dịch: 8/11/2026

Tóm tắt

Chúng tôi tự cài đặt bốn bộ phân lớp cho MNIST và một mạng tích chập nhỏ (CNN) làm phần mở rộng: Gaussian Naïve Bayes, hồi quy softmax huấn luyện bằng Gradient Descent, MLP với lan truyền ngược viết tay và AdaBoost-SAMME trên cây quyết định có trọng số; CNN dùng im2col và gradient viết tay. Phần tính toán lõi dùng NumPy; scikit-learn chỉ để đối chiếu, PyTorch chỉ làm thư viện tensor cho MLP và CNN (không dùng autograd). Mọi mô hình dùng chung một giao thức không rò rỉ dữ liệu: 50 000 ảnh train, 10 000 ảnh validation tách phân tầng từ tập huấn luyện chính thức, và 10 000 ảnh test chỉ dùng để báo cáo. Siêu tham số được chọn trên validation bằng các lượt quét từng tham số, toàn bộ nhật ký được giữ lại.

Trên tập test, độ chính xác lần lượt là 81.32% (Naïve Bayes), 92.52% (softmax), 98.24% (MLP) và 97.60% (AdaBoost). MLP hơn AdaBoost có ý nghĩa thống kê trên cùng 10 000 ảnh (McNemar chính xác, $p = 4 \times 10^{-6}$), và CNN nhỏ đạt 98.80% với khoảng một phần mười số tham số của MLP. AdaBoost mất 817.9 s để huấn luyện, hơn bảy mươi lần so với MLP (11.0 s), và chậm nhất khi dự đoán từng ảnh (18.705 ms). Kết quả của bản tự cài đặt lệch scikit-learn tối đa 0.66 điểm phần trăm trên cùng cấu hình. Chúng tôi còn chạy lại giao thức trên Fashion-MNIST và EMNIST Digits, làm thí nghiệm đối chứng về năm phép tăng cường dữ liệu, và xây một trình khám phá chạy trong trình duyệt vẽ đúng phép tính của mô hình đã triển khai. Mô hình được xuất sang ONNX, đối chiếu số học với bản Python và chạy trực tiếp trong trình duyệt. Chúng tôi cũng nêu những giới hạn của kết quả: một seed cho mô hình chính, quét tuần tự chưa xét tương tác giữa các siêu tham số, và thời gian đo trên một máy.

Từ khóa: MNIST, Fashion-MNIST, Naïve Bayes, hồi quy softmax, MLP, CNN, AdaBoost, SAMME, tăng cường dữ liệu, ONNX.

Mục lục

1 Giới thiệu **3**

2 Dữ liệu và tiền xử lý **3**

3 Cơ sở lý thuyết và cài đặt **4**
3.1 Gaussian Naïve Bayes 4
3.2 Hồi quy logistic đa lớp (softmax) huấn luyện bằng Gradient Descent 4
3.3 Mạng nơ-ron nhiều lớp (MLP) và lan truyền ngược 5
3.4 AdaBoost đa lớp (SAMME) với cây quyết định có trọng số 5
3.5 Mạng tích chập nhỏ (CNN) 6
3.6 Độ phức tạp tính toán 6

4 Thiết kế thí nghiệm **6**
4.1 Quy trình đánh giá thống nhất 6
4.2 Chiến lược tìm siêu tham số 7
4.3 Phần cứng, đo thời gian và theo dõi thí nghiệm 7

5 Kiểm chứng tính đúng đắn **7**

6 Kết quả **8**
6.1 Siêu tham số được chọn 9
6.2 Độ chính xác và các độ đo 10
6.3 Thời gian 11
6.4 So sánh công bằng với scikit-learn 11
6.5 Đường cong học 12

7 Thảo luận **12**
7.1 Độ chính xác, thời gian và khả năng giải thích . . . 12
7.2 Những chữ số bị nhầm . . . 13
7.3 Giới hạn của kết quả . . . 14

8 Mở rộng: bộ dữ liệu khác, tăng cường dữ liệu, CNN và trình khám phá thực thi **14**
8.1 Fashion-MNIST và EMNIST Digits . . . 14
8.2 Tăng cường dữ liệu: thí nghiệm có đối chứng . . . 15
8.3 CNN nhỏ so với MLP . . . 18
8.4 Trình khám phá thực thi mô hình trên trình duyệt . . . 20
8.5 Trực quan hóa tính toán không phải giải thích quyết định . . . 20
8.6 Theo dõi thí nghiệm và công cụ tìm siêu tham số . . . 21
8.7 Bản demo web và quy trình triển khai . . . 21

9 Kết luận **21**

10 Đóng góp của nhóm và công bố sử dụng AI **22**
10.1 Phân công và đóng góp . . . 22
10.2 Công bố sử dụng công cụ AI . . . 22

A Các lượt quét siêu tham số còn lại **24**

B Tái lập kết quả **25**

1 Giới thiệu

Nhận dạng chữ số viết tay trên MNIST [7] là bài toán phân lớp 10 lớp kinh điển của nhận dạng mẫu: mỗi ảnh xám 28×28 chứa một chữ số thuộc $\{0, 1, \ldots, 9\}$ đã được căn giữa. Đồ án này *tự cài đặt* bốn bộ phân lớp thuộc bốn họ phương pháp khác nhau, cộng một mạng tích chập nhỏ làm phần mở rộng, – mô hình sinh (Gaussian Naïve Bayes), mô hình tuyến tính phân biệt (hồi quy softmax huấn luyện bằng Gradient Descent), mạng nơ-ron nhiều lớp (MLP, lan truyền ngược) và ensemble (AdaBoost-SAMME) – rồi so sánh chúng trên cùng một giao thức đánh giá về độ chính xác, thời gian huấn luyện, thời gian dự đoán và khả năng giải thích.

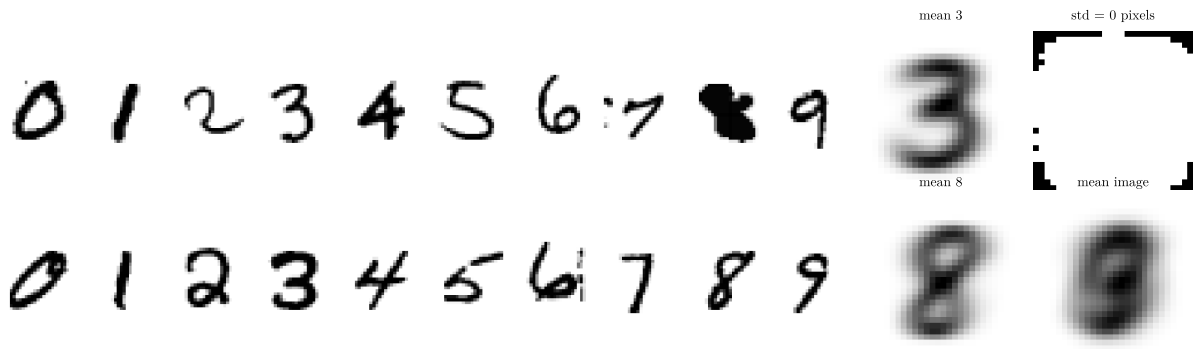
Quy ước "from scratch". Phần tính toán lõi của mỗi thuật toán (ước lượng tham số, hàm mất mát, gradient, cập nhật, dựng cây, cập nhật trọng số boosting) do nhóm viết bằng NumPy. scikit-learn chỉ được dùng để *đối chiếu* (kiểm thử và so sánh baseline); PyTorch chỉ cung cấp tensor và sinh số ngẫu nhiên cho MLP và CNN, *không dùng autograd,* `nn.Module`*,* `optim`*,* `F.conv2d` *hay* `F.max_pool2d` *để huấn luyện.*

Đóng góp chính. (i) Bốn bộ phân lớp với giao diện thống nhất, có kiểm thử đối chiếu scikit-learn và kiểm tra gradient bằng sai phân hữu hạn; (ii) một giao thức thực nghiệm nhất quán, không rò rỉ dữ liệu, điều khiển bằng dòng lệnh và ghi nhận bằng Weights & Biases; (iii) các thí nghiệm siêu tham số với đầy đủ nhật ký, so sánh công bằng với scikit-learn; (iv) bản demo web chạy hoàn toàn trên trình duyệt (ONNX Runtime Web) và quy trình xuất/kiểm chứng mô hình; (v) các nghiên cứu mở rộng có đối chứng: Fashion-MNIST và EMNIST Digits, năm phép tăng cường dữ liệu, CNN so với MLP; (vi) một trình khám phá thực thi trên trình duyệt, dựng từ chính mô hình đã triển khai thông qua một hợp đồng vết có kiểm chứng. Toàn bộ mã nguồn, kết quả và notebook nằm trong kho GitHub của đồ án; mô hình đã huấn luyện được đưa lên Hugging Face Hub (Mục 8.7).

Bố cục. Mục 2 mô tả dữ liệu; Mục 3 trình bày lý thuyết và cài đặt; Mục 4 giao thức thực nghiệm; Mục 5 kiểm chứng; Mục 6 kết quả; Mục 7 thảo luận; các mục cuối là mở rộng, kết luận, đóng góp nhóm và công bố việc sử dụng AI.

2 Dữ liệu và tiền xử lý

MNIST gồm 60 000 ảnh huấn luyện và 10 000 ảnh kiểm thử chính thức, mỗi ảnh 28×28 điểm ảnh 8 bit (0 = nền đen, 255 = nét trắng), chữ số đã được chuẩn hóa kích thước về hộp 20×20 và căn giữa theo trọng tâm [7]. Dữ liệu được tải bằng `torchvision.datasets.MNIST`.



Hình 1. Khảo sát dữ liệu (tập huấn luyện chính thức): hai hàng mẫu ngẫu nhiên cho mỗi chữ số, ảnh trung bình của lớp 3 và 8, bản đồ pixel không đổi, ảnh trung bình toàn tập.

Phân bố lớp. Các lớp gần cân bằng: lớp 1 chiếm nhiều nhất (khoảng 11,2%) và lớp 5 ít nhất

(khoảng 9,0%), nên accuracy là chỉ số hợp lý, nhưng vẫn báo cáo macro precision/recall/F1 để lớp ít mẫu không bị che khuất.

Pixel không mang thông tin. Khoảng 81% giá trị điểm ảnh là nền (0). Trên 50 000 ảnh huấn luyện có 71 vị trí *luôn* là nền (phương sai bằng 0), tập trung ở viền ảnh. Chúng làm Naïve Bayes có phương sai 0 (cần smoothing, Mục 3) và không bao giờ được cây quyết định chọn để tách.

Chia tập và chuẩn hóa. Như trong Mục 4: chia phân tầng 50 000/10 000 (seed 42, mã băm `52cc8206ff96f56d`), chuẩn hóa vô hướng với $\bar{x} = 0{,}13063$ và $s = 0{,}30808$ (tính trên phần train). Dùng một cặp $(\bar{x}, s)$ chung cho cả ảnh (thay vì chuẩn hóa từng pixel) giữ nguyên hình dạng của dữ liệu, tránh chia cho phương sai gần 0 của các pixel ở viền và cho phép bản demo trên trình duyệt áp dụng đúng cùng phép biến đổi. Ảnh được làm phẳng thành vector 784 chiều.

3 Cơ sở lý thuyết và cài đặt

Ký hiệu: $\boldsymbol{x} \in \mathbb{R}^D$ ($D = 784$) là ảnh đã chuẩn hóa, $y \in \{0, \dots, K-1\}$ ($K = 10$) là nhãn, tập huấn luyện $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^N$. Mọi thuật toán dưới đây có *phần tính toán lõi do nhóm tự viết* bằng NumPy (và PyTorch chỉ như thư viện tensor cho MLP, không dùng autograd); scikit-learn chỉ xuất hiện trong các bài kiểm thử và phần đối chiếu (Mục 5). Giao diện chung (`fit`, `predict`, `predict_proba`, `save`/`load`) nằm trong `src/models/base.py`.

3.1 Gaussian Naïve Bayes

Mô hình sinh với giả thiết độc lập có điều kiện giữa các pixel khi biết lớp, mỗi pixel theo phân phối chuẩn [2, 4]:

$$p(\boldsymbol{x} \mid y = k) = \prod_{j=1}^{D} \mathcal{N}\!\left(x_j; \mu_{kj}, \sigma_{kj}^2\right), \qquad \hat{y} = \arg\max_k \Big[\log \pi_k - \tfrac{1}{2}\sum_{j=1}^{D}\Big(\log(2\pi\sigma_{kj}^2) + \tfrac{(x_j-\mu_{kj})^2}{\sigma_{kj}^2}\Big)\Big]. \tag{1}$$

Ước lượng hợp lý cực đại (MLE) cho ra $\pi_k = N_k/N$, μ_{kj} là trung bình mẫu và σ_{kj}^2 là phương sai mẫu (chia cho N_k) của pixel j trong lớp k. Các pixel luôn bằng 0 có phương sai 0 làm $\log \sigma^2 \to -\infty$, nên cộng thêm $\varepsilon = \texttt{var_smoothing} \cdot \max_j \mathrm{Var}(x_j)$ vào mọi phương sai (cùng quy ước với scikit-learn). Việc tính trong không gian log tránh underflow khi nhân 784 thừa số; xác suất hậu nghiệm được chuẩn hóa bằng log-sum-exp. Huấn luyện chỉ cần một lượt qua dữ liệu, $O(ND)$; dự đoán $O(NKD)$. Siêu tham số duy nhất là `var_smoothing`.

3.2 Hồi quy logistic đa lớp (softmax) huấn luyện bằng Gradient Descent

Với $\boldsymbol{z} = W^\top \boldsymbol{x} + \boldsymbol{b}$, $p_k = \exp(z_k)/\sum_m \exp(z_m)$, hàm mất mát trên một mini-batch $\mathcal{B}$ gồm cross-entropy và phạt L2 (không phạt bias):

$$L(W, \boldsymbol{b}) = -\frac{1}{|\mathcal{B}|}\sum_{i \in \mathcal{B}} \log p_{y_i}(\boldsymbol{x}_i) + \frac{\lambda}{2}\|W\|_F^2. \tag{2}$$

Đạo hàm giải tích: $\partial L/\partial z_k = p_k - \mathbb{1}[y = k]$ nên với $X \in \mathbb{R}^{B \times D}$, P là ma trận xác suất và Y là ma trận one-hot,

$$\nabla_W L = \tfrac{1}{B} X^\top (P - Y) + \lambda W, \qquad \nabla_{\boldsymbol{b}} L = \tfrac{1}{B}\,\mathbf{1}^\top (P - Y). \tag{3}$$

Softmax và cross-entropy được cài ở dạng ổn định số học (trừ $\max_k z_k$ trước khi lấy mũ, dùng $\log\sum\exp$ nên không bao giờ tính $\log 0$). Thuật toán cập nhật $\theta \leftarrow \theta - \eta \nabla_\theta L$ theo mini-batch với dữ liệu được xáo trộn mỗi epoch; hàm mục tiêu là lồi nên không cần khởi tạo ngẫu nhiên (khởi tạo bằng 0). Siêu tham số: tốc độ học η, kích thước batch, λ, số epoch. Lịch sử loss/accuracy

theo epoch được ghi lại để theo dõi hội tụ.

3.3 Mạng nơ-ron nhiều lớp (MLP) và lan truyền ngược

Kiến trúc $\boldsymbol{a}_0 = \boldsymbol{x}$, $\boldsymbol{z}_l = \boldsymbol{a}_{l-1}W_l + \boldsymbol{b}_l$, $\boldsymbol{a}_l = \mathrm{ReLU}(\boldsymbol{z}_l) \odot \boldsymbol{m}_l$ cho các lớp ẩn, lớp cuối cho logits và softmax; mất mát là cross-entropy trung bình. Trọng số khởi tạo He [6]: $W_l \sim \mathcal{N}(0, 2/\mathrm{fan_in})$, bias bằng 0. Dropout dạng *inverted* [10]: $m_{l,j} \in \{0,\, 1/(1-p)\}$, chỉ bật khi huấn luyện. Lan truyền ngược [9] viết tay (không dùng autograd):

$$\boldsymbol{\delta}_L = \tfrac{1}{B}\big(\mathrm{softmax}(\boldsymbol{z}_L) - Y\big), \tag{4}$$

$$\nabla_{W_l} = \boldsymbol{a}_{l-1}^{\top}\boldsymbol{\delta}_l, \quad \nabla_{\boldsymbol{b}_l} = \textstyle\sum_{\text{hàng}} \boldsymbol{\delta}_l, \qquad \boldsymbol{\delta}_{l-1} = (\boldsymbol{\delta}_l W_l^{\top}) \odot \mathbb{1}[\boldsymbol{a}_{l-1} > 0] \odot \boldsymbol{m}_{l-1}. \tag{5}$$

Cập nhật bằng SGD có momentum [11], viết tường minh: $\boldsymbol{v} \leftarrow \mu\boldsymbol{v} + \boldsymbol{g}$, $\theta \leftarrow \theta - \eta\boldsymbol{v}$ (trùng với `torch.optim.SGD`, được kiểm thử). *Early stopping* [8]: 8% tập huấn luyện được tách nội bộ làm tập dừng, theo dõi cross-entropy (tắt dropout), kiên nhẫn 5 epoch, ngưỡng cải thiện 10^{-4}, khôi phục trọng số tốt nhất; do đó tập validation của đồ án không tham gia vào quá trình huấn luyện. Mỗi lần `fit` khởi tạo mạng và bộ sinh số ngẫu nhiên (`torch.Generator`) riêng theo `seed`, nên kết quả tái lập được.

3.4 AdaBoost đa lớp (SAMME) với cây quyết định có trọng số

Chúng tôi chọn boosting vì nó là thuật toán ensemble thật sự (các bộ học yếu được huấn luyện tuần tự và có trọng số), không phải lấy trung bình các mô hình khác. SAMME [12] mở rộng AdaBoost [5] cho K lớp: khởi tạo $w_i = 1/N$; ở vòng t, huấn luyện cây h_t với trọng số mẫu w, tính sai số có trọng số $\mathrm{err}_t = \sum_i w_i \mathbb{1}[h_t(\boldsymbol{x}_i) \neq y_i]$ (với w đã chuẩn hóa) và

$$\alpha_t = \eta\Big(\ln\frac{1-\mathrm{err}_t}{\mathrm{err}_t} + \ln(K-1)\Big), \tag{6}$$

$$w_i \leftarrow w_i \exp\big(\alpha_t \mathbb{1}[h_t(\boldsymbol{x}_i) \neq y_i]\big) \text{ rồi chuẩn hóa}, \tag{7}$$

$$H(\boldsymbol{x}) = \arg\max_k \sum_t \alpha_t \mathbb{1}[h_t(\boldsymbol{x}) = k]\,. \tag{8}$$

Số hạng $\ln(K-1)$ khiến bộ học chỉ cần tốt hơn đoán ngẫu nhiên ($\mathrm{err} < 1 - 1/K$) thay vì $< 1/2$; vòng lặp dừng khi điều kiện này bị vi phạm. η là hệ số co (`learning_rate`).

Bộ học yếu: cây CART độ sâu $\le d_{\max}$ giảm thiểu tạp chất Gini có trọng số $G(S) = W_S - \sum_k W_{S,k}^2/W_S$ [3], với $W_S = \sum_{i\in S} w_i$, $W_{S,k} = \sum_{i\in S,\, y_i=k} w_i$. Để huấn luyện nhanh trên $50\,000\times784$, mỗi đặc trưng được lượng tử hóa *một lần* thành n_{bins} khoảng đều nhau; ở mỗi tầng của cây, trọng số theo (nút, đặc trưng, bin, lớp) được cộng dồn bằng histogram, tổng tích lũy cho phép đánh giá mọi ngưỡng của mọi đặc trưng trong $O(ND)$ mỗi tầng. Ngưỡng thực được lưu cho dự đoán nên suy luận không cần lượng tử hóa. Tuỳ chọn `max_features` chọn ngẫu nhiên một tập con đặc trưng cho mỗi cây.

Thuật toán: AdaBoost-SAMME (`src/models/adaboost.py`)

1. Lượng tử hóa X thành bin (một lần); $w_i \leftarrow 1/N$.
2. Với $t = 1, \ldots, T$: dựng cây h_t từ histogram có trọng số (chọn cặp (đặc trưng, ngưỡng) tối thiểu $G(L) + G(R)$ ở mỗi nút, dừng khi không giảm được tạp chất hoặc đạt $d_{\max}$).
3. $\mathrm{err}_t \leftarrow \sum_i w_i[h_t(\boldsymbol{x}_i) \neq y_i]$; nếu $\mathrm{err}_t \ge 1 - 1/K$ thì dừng.
4. $\alpha_t \leftarrow \eta\,(\ln\frac{1-\mathrm{err}_t}{\mathrm{err}_t} + \ln(K-1))$.
5. $w_i \leftarrow w_i\, e^{\alpha_t[h_t(\boldsymbol{x}_i)\neq y_i]}$, chuẩn hóa $\sum_i w_i = 1$.
6. Dự đoán: $\hat{y} = \arg\max_k \sum_t \alpha_t[h_t(\boldsymbol{x}) = k]$; "xác suất" là tỉ lệ phiếu bầu có trọng số (không được hiệu chỉnh).

3.5 Mạng tích chập nhỏ (CNN)

CNN là phần mở rộng so sánh với MLP. Kích thước mỗi lớp *không* được gõ cứng trong báo cáo hay trong giao diện: chúng được đọc từ `CNN.describe()` của mô hình đã huấn luyện và từ đồ thị ONNX xuất ra. Với cấu hình được chọn trên validation (`channels`=(16, 32), `kernel_size`=3, `dense`=(64)), đường đi của một ảnh là

$$1{\times}28{\times}28 \xrightarrow{\text{conv } 3{\times}3} 16{\times}26{\times}26 \xrightarrow{\text{ReLU, max-pool 2}} 16{\times}13{\times}13 \xrightarrow{\text{conv } 3{\times}3} 32{\times}11{\times}11 \xrightarrow{\text{ReLU, max-pool 2}} 32{\times}5{\times}5 \xrightarrow{\text{flatten}} 800 \to 64 \to 10,$$

tổng cộng 56 714 tham số (so với 535 818 của MLP). Tích chập được cài đặt bằng *im2col*: mỗi cửa sổ $k{\times}k{\times}C_{\text{in}}$ được trải thành một hàng, nên giá trị tại một vị trí là $\boldsymbol{p}^{\top}\boldsymbol{w} + b$ (nhân từng phần tử rồi cộng), đúng như phép tính mà trình khám phá vẽ ra; max-pool ghi lại vị trí cực đại để chuyển gradient ngược về đúng ô; gradient của kernel là patches$^{\top}\,\partial L/\partial$out và gradient theo đầu vào được cộng dồn bằng *col2im*.

Phần viết tay và phần dùng PyTorch. Viết tay: im2col/col2im, lan truyền tiến và lùi của tích chập, max-pool và ReLU, lớp dày đặc, softmax kèm cross-entropy, cập nhật SGD có momentum, khởi tạo He, dropout, dừng sớm. PyTorch chỉ là kho tensor và phép nhân ma trận (`matmul`); *không* dùng `autograd`, `nn.Module`, `optim`, `F.conv2d` hay `F.max_pool2d`. Tính đúng được kiểm chứng độc lập (Mục 5): tích chập khớp một vòng lặp ngây thơ bảy tầng, và tích chập, max-pool và gradient đầy đủ khớp `F.conv2d`, `F.max_pool2d` và autograd (chỉ dùng trong kiểm thử), cộng thêm sai phân hữu hạn.

3.6 Độ phức tạp tính toán

Mô hình	Huấn luyện	Dự đoán 1 mẫu
Gaussian NB	$O(ND)$	$O(KD)$
Softmax regression	$O(E\,N\,K\,D)$	$O(KD)$
MLP (H nơ-ron ẩn, 1 lớp)	$O(E\,N\,D\,H)$	$O(DH + HK)$
AdaBoost (T cây, độ sâu d)	$O(T\,d\,N\,D)$	$O(T\,d)$
CNN (C_ℓ kênh, kernel k, ảnh $S \times S$)	$O(E\,N\,\sum_\ell S_\ell^2 k^2 C_{\ell-1} C_\ell)$	$O(\sum_\ell S_\ell^2 k^2 C_{\ell-1} C_\ell)$

Bảng 1. Độ phức tạp theo số mẫu N, số chiều D, số lớp K, số epoch E. Suy luận của AdaBoost chỉ so sánh Td ngưỡng nên rất rẻ.

4 Thiết kế thí nghiệm

4.1 Quy trình đánh giá thống nhất

Một giao thức duy nhất được dùng cho cả bốn mô hình (mã trong `src/experiments/`, điều khiển bằng dòng lệnh `python -m src`):

1. **Chia dữ liệu**: 60 000 ảnh huấn luyện chính thức được chia *phân tầng* (giữ tỉ lệ các lớp) và có seed cố định (42) thành 50 000 ảnh *train* và 10 000 ảnh *validation*; 10 000 ảnh *test* chính thức được giữ riêng. Mã băm của phép chia được lưu cùng mọi kết quả để chứng minh tính nhất quán.
2. **Chuẩn hóa**: chia pixel cho 255 rồi chuẩn hóa $x \leftarrow (x - \bar{x})/(s + 10^{-8})$ với $\bar{x}, s$ (vô hướng) *chỉ tính trên phần train*; áp cùng $\bar{x}, s$ cho validation, test và cho bản demo trên trình duyệt. Trong K-fold, thống kê được tính lại trong từng fold nên không rò rỉ dữ liệu giữa các fold (có kiểm thử).

3. **Chọn siêu tham số** chỉ dựa trên validation (Mục 4.2). Hàm tìm kiếm thậm chí không nhận mảng test (kiểm thử tự động xác nhận).
4. **Đánh giá cuối**: huấn luyện cấu hình đã chọn trên 50 000 ảnh train, báo cáo accuracy, precision/recall/F1 macro và ma trận nhầm lẫn trên train, validation và test. Test chỉ để báo cáo, không dùng để chọn gì.

Các chỉ số được tính từ ma trận nhầm lẫn C (hàng = nhãn thật): $\mathrm{TP}_k = C_{kk}$, $\mathrm{FP}_k = \sum_i C_{ik} - \mathrm{TP}_k$, $\mathrm{FN}_k = \sum_j C_{kj} - \mathrm{TP}_k$; precision, recall, F_1 từng lớp rồi lấy trung bình không trọng số (macro), mẫu số bằng 0 cho giá trị 0. Phần cài đặt này được đối chiếu với scikit-learn trong kiểm thử.

4.2 Chiến lược tìm siêu tham số

Notebook ban đầu định nghĩa lưới MLP gồm 960 tổ hợp × 4 fold (3 840 lần huấn luyện), quá tốn kém và chưa từng hoàn tất. Thay vào đó chúng tôi dùng *quét tuần tự từng siêu tham số* (coordinate descent một vòng): xuất phát từ cấu hình mặc định, lần lượt quét từng siêu tham số trong khi giữ các tham số khác ở giá trị tốt nhất hiện có, chọn giá trị cho validation accuracy cao nhất (hòa thì chọn giá trị đứng trước). Chi phí là $\sum_h |V_h|$ lần huấn luyện thay vì $\prod_h |V_h|$, và mỗi lần quét đúng là đồ thị "đổi một siêu tham số, giữ nguyên các yếu tố khác" mà đề bài yêu cầu. Số vòng boosting và số epoch của softmax được chọn từ *đường cong theo vòng/epoch* của một lần huấn luyện dài (cùng quỹ đạo nên tương đương huấn luyện lại). Mọi cấu hình đã chạy đều được ghi vào `results/tuning/*.json` cùng thời gian; không có kết quả nào được nhập tay. Hạn chế: coordinate descent có thể bỏ sót tương tác giữa các siêu tham số và mỗi cấu hình chỉ chạy một seed.

4.3 Phần cứng, đo thời gian và theo dõi thí nghiệm

Mọi thời gian được đo bằng `time.perf_counter` trên *CPU* (13th Gen Intel Core i7-13620H, 16 luồng logic, PyTorch 8 luồng, Python 3.12.14), không dùng GPU. Thời gian huấn luyện là thời gian chạy thực của `fit` trên 50 000 ảnh; thời gian dự đoán là trung vị của 5 lần dự đoán trọn tập 10 000 ảnh và độ trễ trung vị của 200 lần dự đoán từng ảnh riêng lẻ. Các lần chạy được ghi bằng Weights & Biases [1] (tên mô hình, siêu tham số, seed, mã commit Git, mã băm phép chia, loss/accuracy theo epoch, ma trận nhầm lẫn, thời gian, artifact mô hình); khi không có khoá API, chế độ *offline* ghi cục bộ và có thể đồng bộ sau bằng `wandb sync`; chế độ *disabled* vẫn cho kết quả JSON đầy đủ.

5 Kiểm chứng tính đúng đắn

Gồm 107 kiểm thử tự động (`pytest`), chạy trong vài chục giây trên CPU và trong CI (GitHub Actions) mà không cần tải MNIST. Dữ liệu `digits` 8×8 đi kèm scikit-learn thay MNIST trong kiểm thử.

Gaussian NB. Trung bình, phương sai, prior, nhãn dự đoán, xác suất và log-xác suất khớp scikit-learn `GaussianNB` (sai số $< 10^{-6}$) với ba giá trị `var_smoothing`; xác suất chuẩn hóa về 1 và hữu hạn ngay cả khi đầu vào cực lớn; pixel phương sai 0 không gây lỗi; kiểm tra đầu vào (NaN, sai kích thước, `var_smoothing` < 0).
Softmax regression. Gradient giải tích khớp sai phân hữu hạn trung tâm ($\epsilon = 10^{-6}$, sai số $< 10^{-7}$) có và không có L2; softmax bất biến theo phép dịch và ổn định với logits ± 1000; loss giảm đơn điệu trên dữ liệu thật; khi cả hai hội tụ, nghiệm và xác suất trùng với `LogisticRegression` (L-BFGS) của scikit-learn trên cùng hàm mục tiêu (giá trị mục tiêu chênh $< 10^{-5}$).
MLP. Trên mạng nhỏ độ chính xác kép: gradient lan truyền ngược khớp sai phân hữu hạn (không dropout và có dropout với mặt nạ cố định), khớp autograd của PyTorch (dùng *chỉ trong kiểm thử*), đạo hàm ReLU bằng 0 ở nơ-ron không hoạt động, bước SGD momentum trùng `torch.optim.SGD`,

độ lệch chuẩn khởi tạo He đúng $\sqrt{2/\text{fan_in}}$, mặt nạ dropout là inverted và chỉ có khi huấn luyện, cùng seed cho trọng số trùng bit, early stopping không dùng tập validation ngoài, dự đoán khớp một bản cài đặt forward độc lập bằng NumPy, lưu/nạp cho kết quả y hệt.

AdaBoost. Cây một nút và nhiều tầng cho phân hoạch giống `DecisionTreeClassifier` (Gini, `sample_weight`) trên dữ liệu lưới nguyên; trọng số mẫu quyết định nhãn lá; công thức $\alpha_t = \ln \frac{1-\text{err}_t}{\text{err}_t} + \ln(K-1)$ được kiểm tra đúng; độ chính xác tăng khi thêm cây; trên `digits` độ chính xác chênh lệch `AdaBoostClassifier` (SAMME) không quá 6 điểm phần trăm với cùng số cây và độ sâu (hai bản cài đặt khác nhau về lượng tử hóa ngưỡng và tie-breaking nên không thể trùng tuyệt đối).

CNN. Tích chập (im2col) khớp một vòng lặp ngây thơ bảy tầng; tích chập và max-pool khớp `F.conv2d`, `F.max_pool2d`; gradient của toàn mạng khớp autograd của PyTorch và sai phân hữu hạn (dùng *chỉ trong kiểm thử*); `describe()` trả về đúng hình dạng đầu ra từng lớp; lưu/nạp cho kết quả y hệt.

Bundle, hợp đồng vết và trình duyệt. Với từng bộ dữ liệu và mô hình: ONNX khớp Python trên toàn bộ ảnh test; các đầu ra trung gian khai báo trong `trace.runtime_outputs` đúng bằng các đầu ra của đồ thị; hình dạng trong `trace.layers` bằng hình dạng của ONNX; kiến trúc ghi trong manifest khớp topology của checkpoint; `params.bin` trùng từng bit với các initializer của ONNX; phép tái dựng (tổng có trọng số của MLP, tích chập và gộp của CNN, phân rã log-likelihood của Naïve Bayes, $x_i W_{ik}$ của softmax, đường đi và phiếu của AdaBoost) khớp đầu ra của đồ thị. Phía trình duyệt, kiểm thử Vitest chạy ONNX Runtime Web và bộ đánh giá AdaBoost trên 30 ảnh mẫu cho mỗi bộ dữ liệu và so với kết quả Python, kiểm tra mã băm SHA-256, kiểm tra kết quả cũ không còn hiển thị khi đổi mô hình hay ảnh; kiểm thử đầu-cuối bằng Playwright điều khiển trình duyệt thật (vẽ, chọn mô hình, nhìn vào bên trong, bước qua phép tính). Các kiểm thử này chạy trên đúng bundle được ghim ở `web/model-lock.json` và *thất bại* (không bỏ qua) khi thiếu bundle.

Tiền xử lý, chỉ số, pipeline. Phép chia phân tầng là một phân hoạch, giữ tỉ lệ lớp, tái lập theo seed; thống kê chuẩn hóa chỉ từ phần train; K-fold không rò rỉ; chỉ số khớp scikit-learn trên dữ liệu ngẫu nhiên và khớp ví dụ tính tay; pipeline huấn luyện/đánh giá/xuất mô hình tái lập kết quả; ONNX khớp mô hình Python (Mục 8.7).

6 Kết quả

Mọi số liệu trong mục này được đọc từ các tệp kết quả do chương trình ghi ra (`results/final`, `results/tuning`, `results/baselines`); không có con số nào nhập tay. Cấu hình của từng mô hình là cấu hình tốt nhất trên validation (bảng 2); mỗi mô hình được huấn luyện lại một lần trên 50 000 ảnh train rồi mới chạm vào tập test.

6.1 Siêu tham số được chọn

Mô hình	Siêu tham số được chọn	Số cấu hình đã chạy
Gaussian NB	`var_smoothing=0.1`	12
Softmax regression	`learning_rate=0.03, l2=0.0, epochs=67, batch_size=128`	16
MLP	`hidden_sizes=(512, 256), learning_rate=0.05, momentum=0.9, p_drop=0.0, batch_size=128, epochs=40`	22
AdaBoost-SAMME	`n_estimators=393, max_depth=10, learning_rate=0.1, n_bins=32, max_features=0.5`	14
CNN	`channels=(16, 32), kernel_size=3, dense=(64,), learning_rate=0.01, momentum=0.9, p_drop=0.2, batch_size=128, epochs=20`	10

Bảng 2. Cấu hình chọn bằng validation. `epochs` của softmax và `n_estimators` của AdaBoost lấy từ đường cong validation của một lần chạy dài (tối đa 100 epoch và 400 vòng).

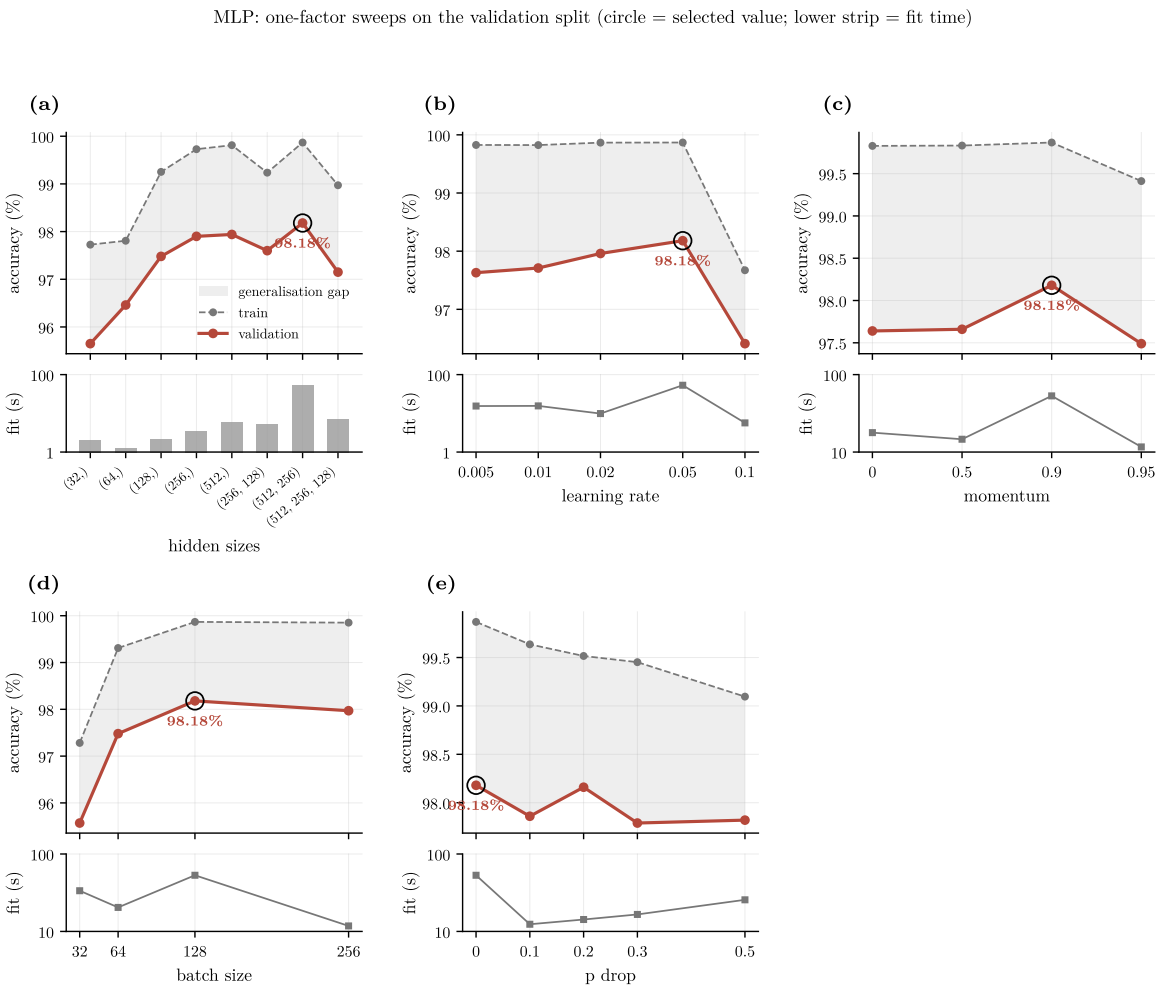
Bảng 3 tóm tắt từng lượt quét: khoảng dao động của validation cho thấy tham số nào thật sự quan trọng. Độ sâu cây quyết định (AdaBoost) và kích thước/hình dạng mạng (MLP) là hai tham số kéo độ chính xác nhiều nhất; với Naïve Bayes, `var_smoothing` mặc định 10^{-9} cho kết quả rất tệ (xem Mục 7).

Mô hình	Siêu tham số	Số giá trị	Val thấp nhất	Val cao nhất	Giá trị tốt nhất
Gaussian NB	`var_smoothing`	12	56.05	79.67	`0.1`
Softmax regression	`learning_rate`	6	88.03	91.72	`0.03`
Softmax regression	`batch_size`	6	90.23	91.72	`128`
Softmax regression	`l2`	5	91.20	91.76	`0.0`
MLP	`hidden_sizes`	8	95.65	98.18	`(512, 256)`
MLP	`learning_rate`	5	96.41	98.18	`0.05`
MLP	`momentum`	4	97.49	98.18	`0.9`
MLP	`batch_size`	4	95.57	98.18	`128`
MLP	`p_drop`	5	97.79	98.18	`0.0`
AdaBoost-SAMME	`max_depth`	5	77.23	95.98	`10`
AdaBoost-SAMME	`max_features`	3	95.98	96.29	`0.5`
AdaBoost-SAMME	`learning_rate`	4	96.29	97.02	`0.1`
AdaBoost-SAMME	`n_bins`	4	96.73	97.02	`32`
CNN	`channels`	3	97.29	97.84	`(16, 32)`
CNN	`kernel_size`	2	97.29	97.84	`3`
CNN	`dense`	2	97.84	98.57	`(64)`
CNN	`learning_rate`	3	11.24	98.61	`0.01`
CNN	`batch_size`	3	98.28	98.61	`128`
CNN	`p_drop`	2	98.61	98.65	`0.2`

Bảng 3. Từng lượt quét siêu tham số trên tập validation (độ chính xác, %). Các lượt quét chạy tuần tự nên mỗi bảng ứng với phần còn lại giữ ở giá trị tốt nhất đã tìm được trước đó.

Hình 2 cho thấy các lượt quét của MLP, kèm khoảng cách giữa train và validation. Hình của ba mô hình còn lại nằm ở Mục A. Các thanh thời gian ở dải dưới chỉ mang tính tham khảo: một số lượt quét có thể chạy cùng lúc với công việc khác trên máy nên thời gian không so sánh được

tuyệt đối với bảng thời gian cuối.



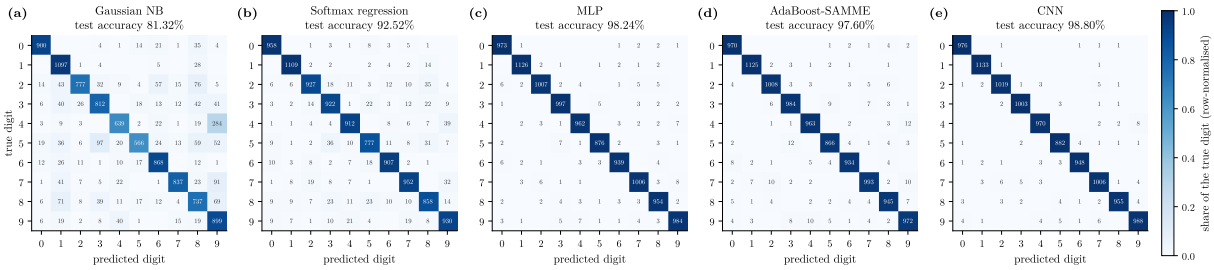
Hình 2. MLP: quét từng siêu tham số trên validation. Vòng tròn đánh dấu giá trị được chọn.

6.2 Độ chính xác và các độ đo

Mô hình	Train	Val	Test	P_{macro}	R_{macro}	$F1_{macro}$
Gaussian NB	80.13	79.67	**81.32**	82.98	80.95	81.16
Softmax regression	93.62	92.15	**92.52**	92.45	92.41	92.42
MLP	99.87	98.18	**98.24**	98.23	98.23	98.23
AdaBoost-SAMME	100.00	97.59	**97.60**	97.58	97.58	97.58
CNN	99.40	98.65	**98.80**	98.79	98.79	98.79

Bảng 4. Độ chính xác (%) trên train, validation, test và precision/recall/F1 macro trên test. Một seed (42) cho mỗi mô hình.

Độ chính xác test của bốn mô hình là 81.32, 92.52, 98.24 và 97.60% (bảng 4). Với $n = 10\,000$ ảnh, khoảng tin cậy nhị thức 95% cho độ chính xác quanh 98% rộng khoảng ± 0.3 điểm phần trăm, nên chênh lệch giữa MLP và AdaBoost (98.24% so với 97.60%) không nhỏ hơn nhiễu lấy mẫu một cách hiển nhiên. Vì cả hai được đánh giá trên cùng ảnh nên chúng tôi dùng kiểm định McNemar chính xác: có 127 ảnh chỉ MLP đoán đúng và 63 ảnh chỉ AdaBoost đoán đúng, $p = 4 \times 10^{-6}$. Kết luận này chỉ nói về một seed và một phép chia dữ liệu.

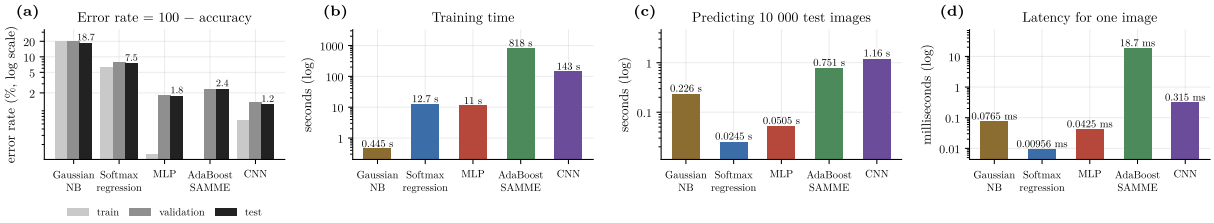


Hình 3. Ma trận nhầm lẫn trên tập test (hàng = chữ số thật, chuẩn hóa theo hàng; số trong ô là số ảnh).

6.3 Thời gian

Mô hình	Huấn luyện (s)	Dự đoán 10 000 mẫu (s)	1 mẫu (ms)
Gaussian NB	0.4	0.226	0.076
Softmax regression	12.7	0.024	0.010
MLP	11.0	0.051	0.042
AdaBoost-SAMME	817.9	0.751	18.705
CNN	143.0	1.163	0.315

Bảng 5. Thời gian trên CPU (cấu hình máy ở Mục 4.3). Huấn luyện trên 50 000 ảnh; dự đoán 10 000 ảnh test; độ trễ là trung vị của 200 lần dự đoán một ảnh.



Hình 4. Lỗi và thời gian của bốn mô hình, thang log. Vạch sáng/trung/đậm trong (a) là train/validation/test; sai số train của AdaBoost bằng 0 nên cột train không hiện trên thang log.

6.4 So sánh công bằng với scikit-learn

Với mỗi mô hình, scikit-learn được dùng *cùng siêu tham số* đã chọn ở trên, huấn luyện trên cùng 50 000 ảnh và đánh giá trên cùng tập test (không chọn lại siêu tham số cho scikit-learn). Đây là so sánh về tính đúng đắn của cài đặt hơn là cuộc đua về thuật toán.

Mô hình	Test (ours)	Test (sklearn)	$\lvert\Delta\rvert$	Fit ours (s)	Fit sklearn (s)
Gaussian NB	81.32	81.32	0.00	0.4	0.2
Softmax regression	92.52	91.86	0.66	12.7	154.6
MLP	98.24	98.02	0.22	11.0	57.8
AdaBoost-SAMME	97.60	97.50	0.10	817.9	1892.8

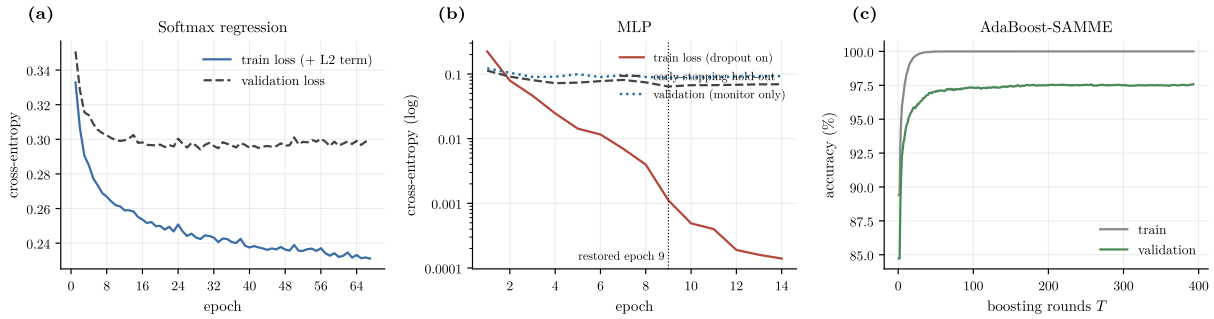
Bảng 6. Bản tự cài đặt so với scikit-learn trên tập test (độ chính xác, %) và thời gian huấn luyện.

- **Naïve Bayes** trùng khớp hoàn toàn (81.32% cả hai), đúng như kiểm thử số học ở Mục 5.

- **Softmax**: bản của chúng tôi cao hơn 0.66 điểm phần trăm. Đây không phải lợi thế thuật toán: scikit-learn chạy L-BFGS đến giới hạn 1000 vòng (cảnh báo chưa hội tụ), không dừng sớm và không chính quy hóa ($\ell_2 = 0$ như cấu hình được chọn), còn chúng tôi dừng ở epoch được chọn trên validation.
- **MLP**: 98.24% so với 98.02%. Cùng kiến trúc, momentum, batch và quy tắc dừng sớm, nhưng khởi tạo, thứ tự lấy mẫu và cách tách tập dừng sớm khác nhau nên chênh 0.22 điểm phần trăm nằm trong dao động giữa các seed (xem bảng 9).
- **AdaBoost**: 97.60% so với 97.50% (chênh 0.10). Hai bản không cùng thuật toán chia: của chúng tôi dùng histogram `n_bins`=32 và chọn ngẫu nhiên một nửa số đặc trưng cho mỗi cây, scikit-learn tìm ngưỡng chính xác trên mọi đặc trưng. Thời gian huấn luyện (817.9 s so với 1892.8 s) vì vậy không phải so sánh sạch, và lượt chạy scikit-learn có trùng thời gian với một thí nghiệm khác trên cùng máy.

Thời gian của các bản scikit-learn khác nhau chủ yếu do thư viện đại số tuyến tính và cách cài đặt (ví dụ `lbfgs` lặp 1000 vòng trên toàn bộ dữ liệu), không nên đọc thành nhận xét về thuật toán.

6.5 Đường cong học



Hình 5. (a) softmax: loss train có cộng thành phần ℓ_2 (bằng 0 ở cấu hình được chọn) và loss validation theo epoch. (b) MLP: loss train (có dropout nếu bật), loss trên tập giữ lại để dừng sớm và loss validation chỉ để theo dõi; epoch tốt nhất được khôi phục. (c) AdaBoost: độ chính xác theo số vòng boosting.

7 Thảo luận

7.1 Độ chính xác, thời gian và khả năng giải thích

Bốn mô hình nằm trên một đường đánh đổi khá rõ giữa độ chính xác, chi phí và độ dễ đọc (bảng 4 và 5).

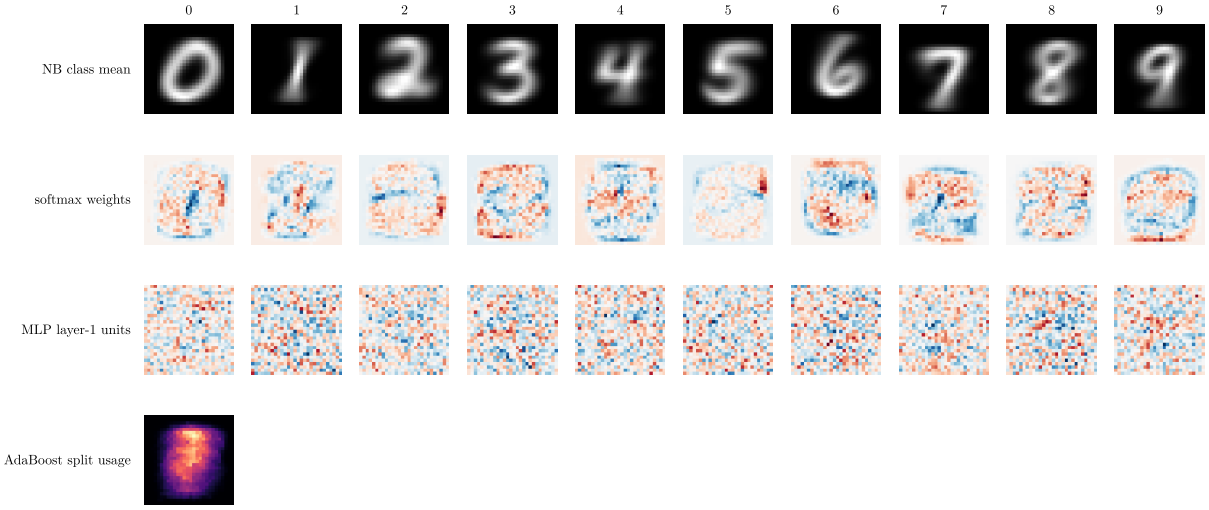
Naïve Bayes (81.32%) rẻ nhất: huấn luyện trong 0.4 s, vì chỉ cần trung bình và phương sai từng pixel theo lớp. Nhưng giả thiết các pixel độc lập có điều kiện theo lớp sai với ảnh chữ số, nơi các pixel lân cận tương quan mạnh. Hệ quả thấy rõ ở bước chọn siêu tham số: với `var_smoothing` mặc định 10^{-9} validation chỉ đạt khoảng 56%, và tăng lên 0.1 mới đạt 79.67%. Giải thích hợp lý (chúng tôi chưa kiểm chứng riêng): nhiều pixel ở rìa ảnh gần như luôn bằng 0 nên phương sai ước lượng rất nhỏ, và một pixel hiếm khi sáng lên cũng đủ kéo log-likelihood của lớp đó xuống rất thấp; làm trơn phương sai bù lại điều này. Đổi lại, mô hình dễ giải thích nhất: ảnh trung bình của từng lớp (hình 6, hàng đầu) cho biết nó "nhìn thấy" gì.

Softmax (92.52%) là mô hình tuyến tính: mỗi lớp là một mẫu (template) w_k và lớp nào có điểm $w_k^\top x + b_k$ cao nhất thì thắng. Nó nhanh (12.7 s huấn luyện, 0.010 ms mỗi ảnh) nhưng bị giới hạn bởi biên quyết định tuyến tính. Trọng số (hình 6, hàng hai) đọc được ở mức mờ: có vệt dương/âm theo nét của chữ số, song nhiễu hơn ảnh trung bình của Naïve Bayes.

MLP (98.24%) là mô hình tốt nhất và vẫn rẻ: 11.0 s huấn luyện vì dừng sớm ở epoch 14 (khôi

phục epoch 9). Đổi lại, các đơn vị lớp ẩn (hình 6, hàng ba) gần như không đọc được bằng mắt. Chênh lệch giữa train (99.87%) và test (98.24%) khoảng 1.63 điểm phần trăm; dropout không giúp thêm ở lượt quét (khoảng `p_drop`=0.2 chỉ sát giá trị 0, trong phạm vi nhiễu).

AdaBoost (97.60%) đạt độ chính xác cao nhưng đắt nhất mọi mặt: 817.9 s huấn luyện, 0.751 s cho 10 000 ảnh test, 18.705 ms cho một ảnh vì phải duyệt 393 cây, và tệp xuất ra nặng hơn 15 MiB. Độ chính xác train đạt 100.00% từ rất sớm (hình 5c) trong khi validation vẫn tăng chậm theo số vòng; đây là hành vi quen thuộc của boosting với cây sâu: sai số huấn luyện về 0 không có nghĩa là hết khả năng tổng quát hóa. Mô hình có một dạng giải thích riêng: tần suất các pixel được dùng để chia nhánh (hình 6, hàng cuối) tập trung ở vùng trung tâm, nơi có nét chữ. Điểm số của nó là tỉ lệ phiếu, *chưa hiệu chỉnh* thành xác suất.



Hình 6. Cái mà mỗi mô hình "học": ảnh trung bình theo lớp của Naïve Bayes, trọng số softmax (đỏ dương, xanh âm), một số đơn vị ẩn đầu tiên của MLP và bản đồ tần suất chia nhánh của AdaBoost.

7.2 Những chữ số bị nhầm

Cặp nhầm nhiều nhất của từng mô hình (nhãn thật→nhãn dự đoán, số ảnh) trong tập test:

Gaussian NB. 4→9 (284), 5→3 (97), 7→9 (91).

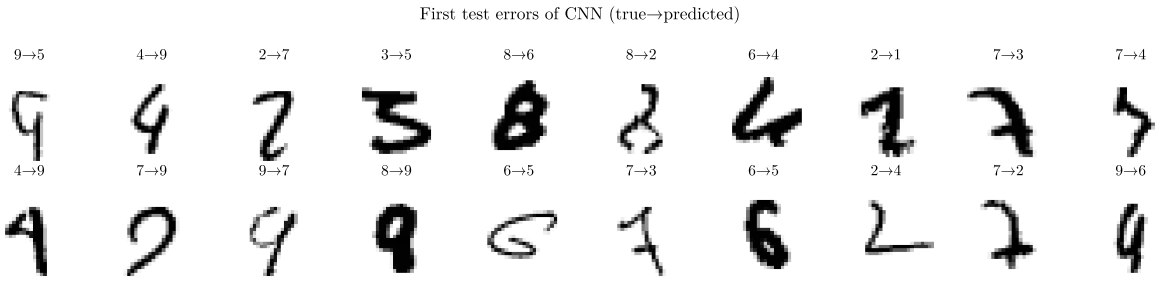
Softmax regression. 4→9 (39), 5→3 (36), 2→8 (35).

MLP. 7→9 (8), 9→4 (7), 5→3 (7).

AdaBoost-SAMME. 5→3 (12), 4→9 (12), 7→9 (10).

CNN. 4→9 (8), 9→7 (6), 7→2 (6).

Cả bốn mô hình đều nhầm $4 \to 9$ hoặc $7 \to 9$ và $5 \to 3$: những cặp có nét chung (đầu chữ 4 và chữ 9 khép hay hở, nét ngang của 7). Với Naïve Bayes riêng cặp $4 \to 9$ chiếm tới 284 trong 1868 lỗi. Với MLP, chỉ 176 ảnh sai, và một số ảnh sai là những chữ khó đọc ngay cả với người (hình 7); nên một phần lỗi còn lại có thể đến từ chính nhãn và nét chữ mơ hồ chứ không chỉ từ mô hình.



Hình 7. Hai mươi ảnh test đầu tiên MLP dự đoán sai (nhãn thật→dự đoán).

7.3 Giới hạn của kết quả

- **Một seed.** Mỗi mô hình chính chỉ được huấn luyện một lần (seed 42) và chỉ có một phép chia train/validation. Riêng MLP chúng tôi có ba seed trong thí nghiệm tăng cường (bảng 9): độ lệch chuẩn của test accuracy giữa các seed là cỡ vài phần mười điểm phần trăm, tức cùng bậc với chênh lệch giữa MLP và scikit-learn.
- **Quét tuần tự.** Coordinate descent một vòng không xét tương tác giữa các siêu tham số (ví dụ learning rate tối ưu phụ thuộc kích thước batch và momentum) và có thể bỏ lỡ tổ hợp tốt hơn. Thứ tự quét cũng ảnh hưởng đến kết quả.
- **Chọn trên validation, báo cáo trên test.** Test chỉ được dùng một lần cho mỗi cấu hình cuối cùng. Với Naïve Bayes, test cao hơn validation (81.32% so với 79.67%), điều thường thấy ở MNIST vì tập test chính thức dễ hơn một chút; chúng tôi không dùng điều này để chọn gì.
- **Một máy, CPU, có nhiễu.** Thời gian đo trên một máy; không dùng GPU. Một số lượt chạy (các baseline scikit-learn, tuning) có thể trùng thời gian với công việc khác nên chỉ dùng để so bậc độ lớn.
- **Phạm vi.** Kết quả chỉ nói về MNIST: ảnh 28×28, căn giữa, một chữ số. Bản demo web cắt, thu nhỏ và căn giữa nét vẽ theo cách của MNIST nhưng chữ viết trên màn hình vẫn khác phân phối huấn luyện, nên có thể sai ở những ca MLP không sai trên tập test.

8 Mở rộng: bộ dữ liệu khác, tăng cường dữ liệu, CNN và trình khám phá thực thi

Phần này dùng *cùng mã, cùng giao thức và cùng bộ đánh giá* như các mục trước. Mọi bảng và hình được sinh từ các tệp trong `results/`; các thí nghiệm được chạy lại bằng dòng lệnh (README). Các kết luận chỉ có giá trị cho các bộ dữ liệu và cấu hình đã đo.

8.1 Fashion-MNIST và EMNIST Digits

Cả hai bộ đều là ảnh xám 28×28 với 10 lớp nên dùng lại nguyên pipeline. **Fashion-MNIST** (áo thun, quần, áo len, ..., giày cao cổ) có 60 000 ảnh train và 10 000 ảnh test chính thức. **EMNIST Digits** có 240 000 ảnh train và 40 000 ảnh test chính thức; chúng tôi lấy một tập con phân tầng 60 000 ảnh từ phần train, chia 50 000/10 000 thành train/validation (cùng kích thước như MNIST để thời gian có thể so sánh), giữ nguyên 40 000 ảnh test, và lật/xoay ảnh về chiều đứng như MNIST. Mỗi bộ có phép chia riêng, thống kê chuẩn hóa riêng (chỉ từ train của bộ đó), siêu tham số chọn riêng trên validation của bộ đó bằng quét *rút gọn* (ít giá trị hơn so với MNIST để vừa ngân sách thời gian), và kết quả được báo cáo *tách biệt* với MNIST.

Định nghĩa lớp khác nhau: MNIST và EMNIST Digits đều gán nhãn 0–9 cho chữ số, còn Fashion-MNIST gán 10 loại quần áo cho cùng các chỉ số 0–9. Vì vậy một mô hình huấn luyện trên một bộ *không* được đánh giá trên bộ có định nghĩa lớp khác: chương trình từ chối phép đó. Phép kiểm

tra chéo duy nhất hợp lệ là MNIST → EMNIST Digits, và nó chỉ được báo cáo cho đúng mô hình huấn luyện trên MNIST, tách khỏi mô hình huấn luyện trên EMNIST. Trang web cũng ghi rõ bộ dữ liệu, tên lớp và mô hình đang dùng ở mọi nơi.

Mô hình	MNIST	Fashion-MNIST	EMNIST Digits
Gaussian NB	81.32 / 81.16	67.27 / 66.03	80.03 / 80.13
Softmax regression	92.52 / 92.42	84.27 / 84.18	93.81 / 93.81
MLP	98.24 / 98.23	88.59 / 88.55	98.36 / 98.36
AdaBoost-SAMME	97.60 / 97.58	84.74 / 84.88	97.38 / 97.38
CNN	98.80 / 98.79	88.49 / 88.48	98.64 / 98.64

Bảng 7. Accuracy / macro-F1 trên tập test (%) của từng bộ dữ liệu; mỗi bộ dữ liệu có phép chia train/validation/test riêng, siêu tham số chọn riêng trên validation của bộ đó (quét rút gọn cho Fashion-MNIST và EMNIST Digits). Không so sánh chéo giữa các bộ dữ liệu.

Bảng 7 cho thấy thứ hạng thay đổi theo bộ dữ liệu: các mô hình có cấu trúc (MLP, CNN) vượt xa Naïve Bayes trên Fashion-MNIST, (một cách giải thích hợp lý, chưa được kiểm chứng riêng, là giả thiết các pixel độc lập có điều kiện kém phù hợp hơn với ảnh quần áo), và khoảng cách giữa các mô hình thu hẹp hay nới rộng tùy bộ dữ liệu. Vì siêu tham số của hai bộ mới được chọn bằng quét rút gọn một seed, chênh lệch nhỏ giữa MLP và CNN ở đó không đủ để kết luận mô hình nào tốt hơn. Với EMNIST Digits chúng tôi không chạy baseline scikit-learn (tốn thời gian và không cần cho câu hỏi ở đây), nên bộ này chỉ có kết quả của các bản tự cài đặt.

Kiểm tra chéo MNIST → EMNIST Digits. Đây là phép duy nhất hợp lệ về định nghĩa lớp (Bảng 8). Mô hình huấn luyện trên MNIST tụt mạnh khi gặp EMNIST (Naïve Bayes xuống dưới 50%), trong khi mô hình huấn luyện trên chính EMNIST đạt 80.0% (Naïve Bayes) và 98.6% (CNN); CNN giữ được nhiều nhất. Chúng tôi chưa tách riêng nguyên nhân; một giả thuyết hợp lý là kiểu chữ, kích thước và cách căn giữa của hai bộ khác nhau, và các mô hình có cấu trúc không gian chịu được dịch chuyển tốt hơn. Kết quả này chỉ nói về hai bộ dữ liệu này và không phải bằng chứng về khả năng tổng quát hoá sang chữ viết tay nói chung.

Mô hình	Huấn luyện trên MNIST	Huấn luyện trên EMNIST Digits
Gaussian NB	49.80	80.03
Softmax regression	53.59	93.81
MLP	76.70	98.36
CNN	92.15	98.64
AdaBoost-SAMME	73.69	97.38

Bảng 8. Accuracy (%) trên 40 000 ảnh test của EMNIST Digits. Cột giữa là mô hình huấn luyện trên MNIST (dùng hằng số chuẩn hóa của MNIST) đánh giá trực tiếp; cột phải là mô hình huấn luyện trên EMNIST Digits. Đây là hai mô hình khác nhau.

8.2 Tăng cường dữ liệu: thí nghiệm có đối chứng

Năm phép biến đổi (Hình 8), chỉ áp dụng cho tập train, nhãn không đổi: *xoay* đều trong $[-15°, 15°]$; *dịch* đều trong $[-2, 2]$ pixel theo mỗi trục; *co giãn* đều trong $[0.9, 1.1]$; *nhiễu Gauss* $\sigma = 0.1$ trên thang $[0, 1]$; *nhiễu muối tiêu* với 2% pixel. Các phép hình học được gộp thành một phép affine duy nhất cho mỗi ảnh (nội suy song tuyến tính); đây là biến đổi dữ liệu chứ không phải thành phần của mô hình.

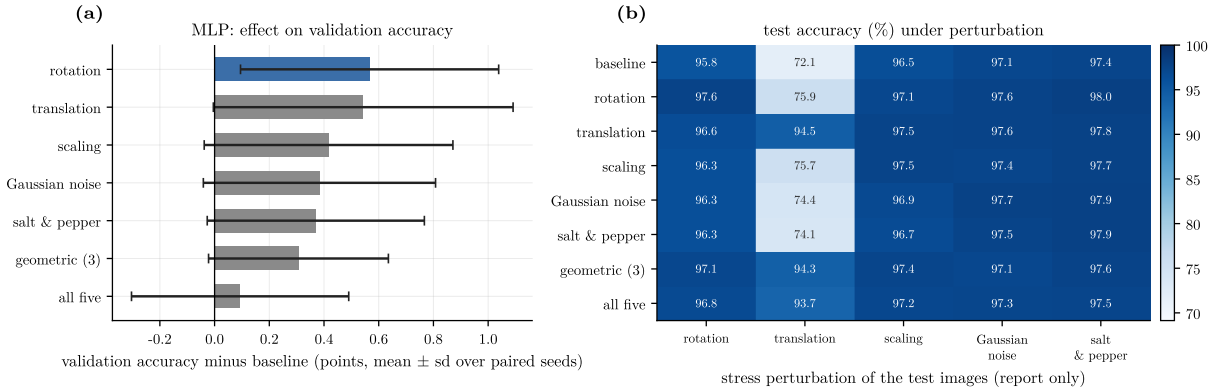
Bố trí thí nghiệm: giữ nguyên siêu tham số, seed và quy tắc dừng sớm; chỉ đổi tập huấn luyện. Điều kiện c thêm một bản sao biến đổi của mỗi ảnh train (100 000 ảnh, gấp đôi điều kiện gốc), nên có *hai yếu tố trộn lẫn*: loại biến đổi và việc có thêm dữ liệu/bước cập nhật. Có tám điều kiện (gốc; từng phép riêng lẻ; nhóm hình học; cả năm), mỗi điều kiện chạy nhiều seed và so sánh *ghép cặp theo seed*. Điều kiện được chọn *chỉ* theo validation accuracy trung bình; test chỉ được báo cáo. Độ bền được đo thêm bằng ảnh test biến đổi cố định (seed 999) ở mức *mạnh hơn* lúc huấn luyện (xoay 20°, dịch 3 pixel, co giãn 0.8–1.2, $\sigma = 0.2$, 4% muối tiêu), dùng chung cho mọi mô hình và chỉ để báo cáo.



Hình 8. Ảnh gốc và từng phép biến đổi ở mức huấn luyện và mức kiểm tra độ bền (cùng các ảnh nguồn).

		Validation		Test	Test sau nhiễu (%)				
Điều kiện	Số mẫu	Acc (%)	Δ so với gốc	Acc (%)	Xoay	Dịch	Co giãn	Gauss	Muối tiêu
Không tăng cường	50 000	97.88 ± 0.42	+0.00 ± 0.00	98.02	95.8	72.1	96.5	97.1	97.4
Xoay	100 000	98.45 ± 0.06	+0.57 ± 0.47	98.48	97.6	75.9	97.1	97.6	98.0
Dịch chuyển	100 000	98.43 ± 0.30	+0.54 ± 0.55	98.41	96.6	94.5	97.5	97.6	97.8
Co giãn	100 000	98.30 ± 0.10	+0.42 ± 0.45	98.33	96.3	75.7	97.5	97.4	97.7
Nhiễu Gauss	100 000	98.27 ± 0.01	+0.38 ± 0.42	98.35	96.3	74.4	96.9	97.7	97.9
Nhiễu muối tiêu	100 000	98.25 ± 0.04	+0.37 ± 0.40	98.32	96.3	74.1	96.7	97.5	97.9
Hình học (xoay, dịch, co giãn)	100 000	98.19 ± 0.21	+0.31 ± 0.33	98.24	97.1	94.3	97.4	97.1	97.6
Cả năm phép	100 000	97.98 ± 0.03	+0.09 ± 0.40	98.11	96.8	93.7	97.2	97.3	97.5

Bảng 9. MLP với cấu hình đã chọn, huấn luyện lại với 3 seed (42, 43, 44) cho mỗi điều kiện; Δ là hiệu validation accuracy so với điều kiện gốc *cùng seed* (trung bình ± độ lệch chuẩn mẫu). 1 bản sao biến đổi cho mỗi ảnh train; điều kiện được chọn chỉ theo validation (`rotation`). Các cột "Test sau nhiễu" đo độ bền trên tập test bị biến đổi ở mức mạnh hơn mức huấn luyện, chỉ để báo cáo.

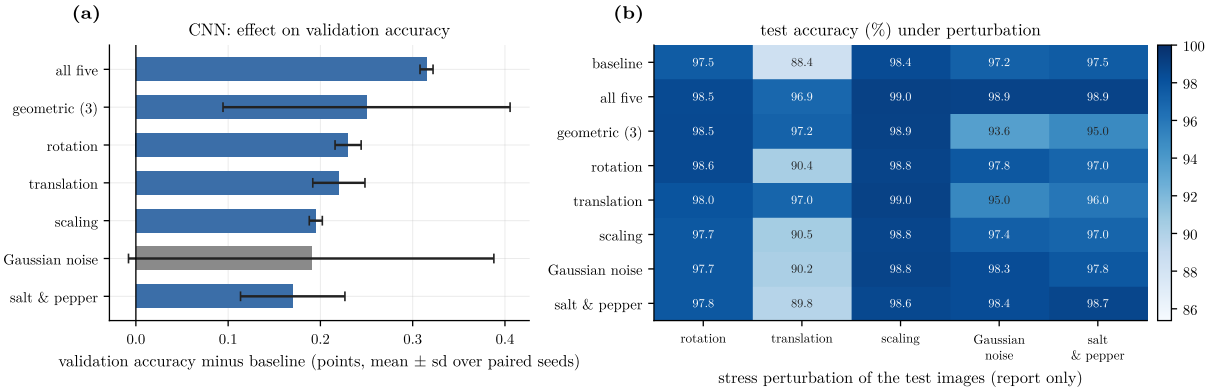


Hình 9. MLP: (a) hiệu validation accuracy so với điều kiện gốc (trung bình ± độ lệch chuẩn của hiệu ghép cặp); (b) accuracy trên ảnh test bị biến đổi mạnh.

Đọc kết quả MLP. Trên ảnh sạch, mọi điều kiện cho validation accuracy cao hơn điều kiện gốc nhưng hiệu ghép cặp có độ lệch chuẩn cùng bậc với chính mức tăng, nên với số seed hiện có chúng tôi *không* khẳng định phép biến đổi nào cải thiện độ chính xác trên ảnh sạch; một phần mức tăng có thể đến từ việc có gấp đôi dữ liệu. Ngược lại, tác dụng *đúng loại* lên độ bền rất rõ: huấn luyện với phép dịch nâng accuracy trên ảnh bị dịch mạnh lên rất nhiều so với điều kiện gốc, huấn luyện với xoay giúp trên ảnh bị xoay, và nhiễu nhẹ giúp ít vì MLP vốn đã khá bền với nhiễu. Cả năm phép cùng lúc không tốt hơn các phép riêng lẻ ở thí nghiệm này, tức là "thêm mọi thứ" không phải lúc nào cũng có lợi. Điều kiện có validation cao nhất được chọn theo quy tắc đã nêu, và mức chênh với điều kiện đứng sau nằm trong độ nhiễu.

		Validation		Test	Test sau nhiễu (%)				
Điều kiện	Số mẫu	Acc (%)	Δ so với gốc	Acc (%)	Xoay	Dịch	Co giãn	Gauss	Muối tiêu
Không tăng cường	50 000	98.67 ± 0.04	+0.00 ± 0.00	98.82	97.5	88.4	98.4	97.2	97.5
Xoay	100 000	98.91 ± 0.05	+0.23 ± 0.01	99.17	98.6	90.4	98.8	97.8	97.0
Dịch chuyển	100 000	98.89 ± 0.06	+0.22 ± 0.03	99.17	98.0	97.0	99.0	95.0	96.0
Co giãn	100 000	98.87 ± 0.03	+0.20 ± 0.01	99.06	97.7	90.5	98.8	97.4	97.0
Nhiễu Gauss	100 000	98.87 ± 0.16	+0.19 ± 0.20	99.06	97.7	90.2	98.8	98.3	97.8
Nhiễu muối tiêu	100 000	98.84 ± 0.02	+0.17 ± 0.06	99.00	97.8	89.8	98.6	98.4	98.7
Hình học (xoay, dịch, co giãn)	100 000	98.92 ± 0.19	+0.25 ± 0.16	99.20	98.5	97.2	98.9	93.6	95.0
Cả năm phép	100 000	98.99 ± 0.03	+0.31 ± 0.01	99.25	98.5	96.9	99.0	98.9	98.9

Bảng 10. CNN với cấu hình đã chọn, huấn luyện lại với 2 seed (42, 43) cho mỗi điều kiện; Δ là hiệu validation accuracy so với điều kiện gốc *cùng seed* (trung bình ± độ lệch chuẩn mẫu). 1 bản sao biến đổi cho mỗi ảnh train; điều kiện được chọn chỉ theo validation (`all`). Các cột "Test sau nhiễu" đo độ bền trên tập test bị biến đổi ở mức mạnh hơn mức huấn luyện, chỉ để báo cáo.



Hình 10. CNN: cùng bố trí như Hình 9.

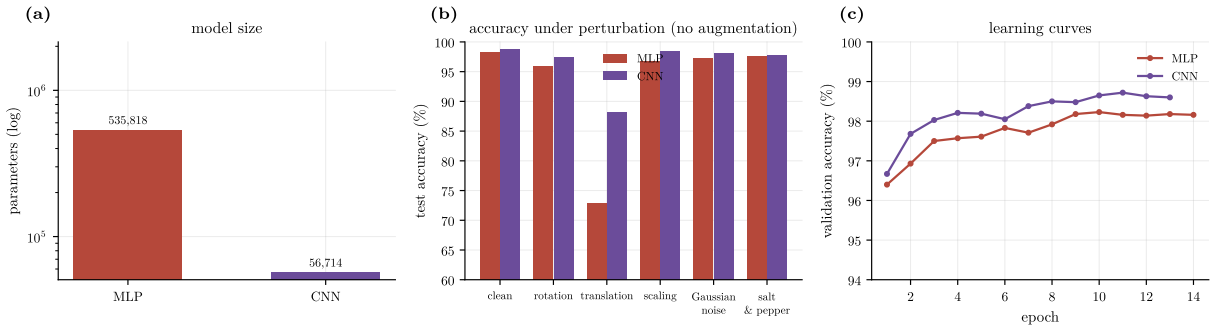
Đọc kết quả CNN. Với CNN, mọi điều kiện đều có validation accuracy cao hơn điều kiện gốc với hiệu ghép cặp (+0.17 … +0.31 điểm, tuỳ điều kiện) lớn hơn độ lệch chuẩn của chúng ở hầu hết điều kiện, và "cả năm phép" có validation cao nhất nên được chọn theo quy tắc. Tuy nhiên chỉ có *hai seed*, độ lệch chuẩn ước lượng từ hai giá trị kém tin cậy, và vẫn có yếu tố trộn lẫn là dữ liệu gấp đôi; vì vậy đây là bằng chứng gợi ý chứ chưa chắc chắn. Có một tác dụng *bất lợi* rõ ràng: huấn luyện chỉ với phép dịch hoặc với nhóm hình học làm CNN *kém bền hơn* với nhiễu Gauss và muối tiêu so với điều kiện gốc (ví dụ trên ảnh bị nhiễu Gauss mạnh, accuracy là 97.2% ở điều kiện gốc nhưng chỉ 95.0% khi huấn luyện với phép dịch và 93.6% với nhóm hình học), trong khi huấn luyện với chính loại nhiễu đó hoặc với cả năm phép thì cải thiện. Tức là một phép tăng cường có thể cải thiện loại biến đổi nó mô phỏng và làm giảm độ bền ở loại khác; chỉ "cả năm phép" tốt đồng thời ở mọi cột cho CNN, còn với MLP thì không.

8.3 CNN nhỏ so với MLP

Cấu hình CNN (Mục 3.5) được chọn bằng quét tuần tự trên validation như các mô hình khác (kênh, kích thước kernel, lớp dày đặc, tốc độ học, batch, dropout); toàn bộ nhật ký nằm trong `results/tuning/`. Bảng 11 và Hình 11 so sánh hai mô hình trên cùng 10 000 ảnh test.

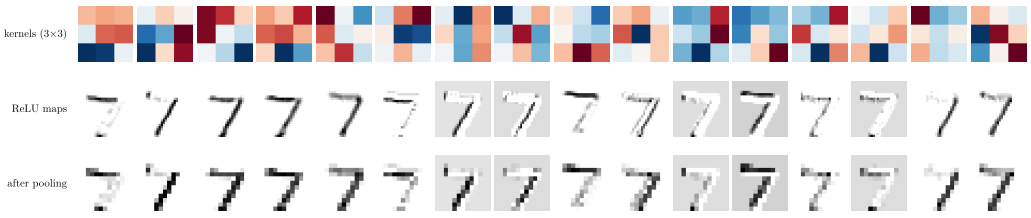
	MLP	CNN
Số tham số	535 818	56 714
Kích thước ONNX (KiB)	2094	223
Accuracy test (%)	98.24	98.80
Precision / Recall / F1 macro (%)	98.23 / 98.23 / 98.23	98.79 / 98.79 / 98.79
Accuracy train − test (điểm)	1.63	0.60
Thời gian huấn luyện (s)	11	143
Độ trễ một ảnh (ms)	0.042	0.315
Accuracy khi xoay (%)	96.00	97.46
Accuracy khi dịch (%)	72.86	88.14
Accuracy khi co giãn (%)	96.71	98.48
Accuracy khi nhiễu Gauss (%)	97.31	98.07
Accuracy khi muối tiêu (%)	97.56	97.71

Bảng 11. CNN và MLP trên MNIST, cùng giao thức; mọi số lấy từ các lần chạy đã ghi. Hàng cuối: ảnh test bị biến đổi cố định (seed 999, mức mạnh hơn lúc huấn luyện), chỉ để báo cáo. Kiểm định McNemar chính xác trên dự đoán test: chỉ MLP đúng 64 ảnh, chỉ CNN đúng 120 ảnh, $p = 4.4e-05$.



Hình 11. CNN và MLP: số tham số, độ bền khi ảnh test bị biến đổi (không dùng tăng cường), đường cong validation.

CNN có accuracy cao hơn MLP (98.80% so với 98.24%) và khác biệt này có ý nghĩa thống kê trên cùng ảnh test (McNemar chính xác, Bảng 11), với *khoảng một phần mười* số tham số, khoảng cách train–test nhỏ hơn và bền hơn rõ rệt khi ảnh bị dịch – đúng điều ta mong đợi từ chia sẻ trọng số và gộp cục bộ. Cái giá là thời gian huấn luyện (143.0 s so với 11.0 s) và độ trễ mỗi ảnh (0.315 ms so với 0.042 ms) cao hơn. Về diễn giải, nơ-ron lớp đầu của MLP có một trọng số cho mỗi pixel nên vẽ được thành ảnh 28×28, còn kernel 3×3 của CNN dùng chung ở mọi vị trí nên dễ vẽ (Hình 12) nhưng một kernel riêng lẻ nói rất ít về quyết định cuối cùng nếu không xét các tầng sau; đây là nhận xét định tính, không phải một điểm số đo được.



Hình 12. Kernel 3×3 lớp đầu đã học, bản đồ ReLU và kết quả sau gộp cho một ảnh test thật. Bản đồ kích hoạt cho thấy lớp *tính ra gì*, không chứng minh vùng nào gây ra quyết định.

8.4 Trình khám phá thực thi mô hình trên trình duyệt

Mục "nhìn vào bên trong" của trang web được làm lại thành một trình khám phá chỉ vẽ những gì *mô hình đang triển khai* thực sự tính, không có mạng giả, kích hoạt giả hay kiến trúc gõ cứng trong giao diện.

Dòng thông tin. (1) Python sinh *hợp đồng vết* `writerhand.trace/1` từ chính đối tượng mô hình đã huấn luyện (`src/export/tracespec.py`): định danh mô hình, đồ thị lớp (DAG với toán tử, đầu vào, hình dạng đầu ra), tiền xử lý, bảng tham số (vị trí trong `params.bin`, hình dạng, mã băm), danh sách đầu ra trung gian cần lấy từ đồ thị, loại đầu ra (xác suất softmax hay tỉ lệ phiếu *chưa hiệu chỉnh*). (2) Bundle gồm manifest (có `bundle_id` và SHA-256 từng tệp), ONNX, `params.bin`, ensemble AdaBoost nhị phân, ảnh test mẫu cùng đầu ra kỳ vọng và bản ghi huấn luyện; nó được đẩy lên Hugging Face và *ghim theo mã commit* trong `web/model-lock.json`; trang web chỉ tải đúng revision đó, kiểm tra SHA-256 manifest so với bản đã commit cùng trang, và kiểm tra từng tệp. (3) Trình duyệt kiểm tra hợp đồng, chạy ONNX Runtime Web với đúng các đầu ra mà hợp đồng khai báo, rồi vẽ sơ đồ *từ metadata*: đổi kiến trúc rồi huấn luyện lại thì sơ đồ tự đổi. (4) Mỗi kết quả thực thi mang dấu (bundle, bộ dữ liệu, mô hình, dấu vân tay của ảnh) và bị loại bỏ khi bất kỳ yếu tố nào đổi, nên không bao giờ hiển thị kết quả cũ.

Từng mô hình. *MLP*: $28 \times 28 \to$ duỗi phẳng 784 $\to$ chuẩn hóa $\to$ các lớp dày đặc với nơ-ron/kết nối chọn được; một tổng có trọng số được tái dựng từ tham số và đặt cạnh giá trị của đồ thị kèm sai lệch tuyệt đối; lớp lớn được rút gọn bằng cách hiển thị các số hạng lớn nhất và ghi rõ đã rút gọn. *CNN*: kernel, bản đồ đặc trưng, cửa sổ trượt với phép nhân–cộng từng pixel, ReLU, cửa sổ gộp và giá trị cực đại, trường tiếp nhận, hình dạng từng tầng và điểm số. *Naïve Bayes*: trung bình, phương sai, số hạng log-likelihood theo lớp và pixel, tổng cộng dồn đối chiếu với log joint của đồ thị. *Softmax*: $x_i W_{ik}$, bias, cộng dồn và đối chiếu với logit. *AdaBoost-SAMME*: các cây thật (pixel, ngưỡng, nhánh, lớp ở lá), α_t, phiếu cộng dồn có thể tạm dừng, bước và kéo; các giá trị này là *tỉ lệ phiếu có trọng số*, không phải xác suất đã hiệu chỉnh. Mọi phép phát có Play/Pause/Step/Back/Replay, tốc độ, chế độ giảm chuyển động và cho xem giá trị không cần dừng.

Hai chế độ tách biệt. *Suy luận* trên ảnh người dùng, và *phát lại huấn luyện* chỉ từ các điểm kiểm tra và số đo *thật* đã ghi lúc huấn luyện lại với đúng cấu hình và seed (`writerhand.replay/1`): mất mát và độ chính xác theo epoch, dự đoán của chính mô hình trên 12 ảnh train cố định ở các epoch được chọn, và một phần trọng số (softmax: toàn bộ W; MLP: 16 nơ-ron đầu của lớp thứ nhất; CNN: mọi kernel). Tệp ghi epoch mà mô hình triển khai tương ứng (`deployed_epoch`, do dừng sớm khôi phục trọng số tốt nhất) và chỉ đánh dấu khớp khi trọng số đã lưu bằng đúng trọng số triển khai. Naïve Bayes không có huấn luyện lặp nên không có phát lại; AdaBoost dùng các tiền tố của chính ensemble đã triển khai.

8.5 Trực quan hóa tính toán không phải giải thích quyết định

Một bản đồ kích hoạt, một kernel hay một đường đi trong cây cho biết mô hình *đã tính gì* với ảnh này; nó không chứng minh rằng điểm ảnh hay bộ lọc đó *gây ra* quyết định, vì các tầng sau có thể bù trừ hoặc bỏ qua. Trong trình khám phá, chỉ những phân rã *chính xác theo cấu trúc mô hình* (số hạng $x_i W_{ik}$ của softmax, đóng góp log-likelihood của Naïve Bayes, phiếu của từng cây) được gọi là phần đóng góp vào điểm số, và vẫn không được gọi là "lý do" theo nghĩa nhân quả; các kích hoạt của MLP/CNN được ghi nhãn là giá trị trung gian. Chúng tôi không dùng phương pháp quy gán (Grad-CAM, tích phân gradient, ...) và không tuyên bố rằng chúng được kiểm chứng.

8.6 Theo dõi thí nghiệm và công cụ tìm siêu tham số

Siêu tham số được tìm bằng *quét tuần tự từng tham số* (coordinate descent) do nhóm tự viết (`src/experiments/search.py`), không dùng Optuna hay thư viện tối ưu siêu tham số: chi phí là tổng số giá trị thay vì tích, và mỗi lượt quét chính là đồ thị "đổi một siêu tham số, giữ nguyên phần còn lại". Nhược điểm là không xét tương tác giữa các siêu tham số. Mọi lần chạy được ghi bằng Weights & Biases ở chế độ *offline* (máy nhóm chưa có khóa API; có thể `wandb sync` sau), bên cạnh nhật ký JSON luôn được ghi trong `results/`.

8.7 Bản demo web và quy trình triển khai

Mô hình được xuất ra một bundle gồm ONNX cho Naïve Bayes, softmax, MLP và CNN (đồ thị dựng tay bằng `onnx.helper`, bước chuẩn hóa $(x - \bar{x})/(s + \varepsilon)$ nằm *trong* đồ thị nên tiền xử lý huấn luyện và suy luận trùng nhau) và một tệp nhị phân cho các cây của AdaBoost (bộ đánh giá viết bằng TypeScript). Mỗi tệp xuất được kiểm tra số học trên toàn bộ ảnh test của bộ dữ liệu tương ứng: Gaussian NB: sai lệch xác suất lớn nhất $1.4e-04$ trùng nhãn 100.00% trên 2 000 ảnh; Softmax regression: sai lệch xác suất lớn nhất $3.4e-06$ trùng nhãn 100.00% trên 2 000 ảnh; MLP: sai lệch xác suất lớn nhất $1.7e-06$ trùng nhãn 100.00% trên 2 000 ảnh; AdaBoost-SAMME: sai lệch xác suất lớn nhất $0.0e+00$ trùng nhãn 100.00% trên 2 000 ảnh; CNN: sai lệch xác suất lớn nhất $7.2e-07$ trùng nhãn 100.00% trên 2 000 ảnh. Bundle cùng model card được đẩy lên Hugging Face Hub (`https://huggingface.co/TranTrinhNhan1111/WriteRHand-MNIST`; không đưa trọng số lên GitHub), và trang demo được triển khai trên GitHub Pages (`https://trantrinhnhan1.github.io/WriteRHand/`). Trang web (React + TypeScript + Vite + ONNX Runtime Web, hoàn toàn tĩnh) cho phép vẽ hoặc tải ảnh, chọn bộ dữ liệu và một trong năm mô hình (chỉ những mô hình có tệp đã kiểm chứng), xem ảnh 28×28 sau tiền xử lý, top-k, bảng kết quả, ma trận nhầm lẫn và đường cong; ảnh vẽ được cắt, thu về hộp 20×20 và căn trọng tâm về $(14, 14)$ đúng như quy trình tạo MNIST. Quy trình CI (GitHub Actions) chạy kiểm tra mã, kiểm thử đơn vị, một lượt chạy thử nhỏ của pipeline, kiểm tra manifest–khóa–revision từ xa khớp nhau, kiểm thử trình duyệt trên bundle đã ghim và build giao diện; không huấn luyện lại ở mỗi commit. Mô hình cho bản demo và các bảng chính vẫn là các mô hình huấn luyện trên dữ liệu gốc (không tăng cường).

9 Kết luận

Nhóm đã tự cài đặt và so sánh bốn bộ phân lớp, cùng một CNN nhỏ làm phần mở rộng, trên MNIST với cùng một giao thức không rò rỉ. CNN cho kết quả tốt nhất (98.80% trên test, với khoảng một phần mười số tham số của MLP, đổi lại huấn luyện và suy luận chậm hơn); trong bốn bộ phân lớp chính, MLP tốt nhất (98.24%) với chi phí nhỏ; AdaBoost-SAMME gần sát (97.60%) nhưng đắt hơn nhiều về thời gian huấn luyện và dự đoán; softmax (92.52%) là lựa chọn nhanh và dễ đọc; Naïve Bayes (81.32%) là mốc đơn giản nhất và bộc lộ rõ cái giá của giả thiết độc lập. Kết quả của các bản tự cài đặt khớp scikit-learn ở mức cho phép của từng thuật toán, và bản xuất ONNX khớp bản Python.

Bài học thực nghiệm lớn nhất: phần lớn lợi ích đến từ *chọn cấu hình bằng validation một cách có kỷ luật* (ví dụ `var_smoothing` cho Naïve Bayes, độ sâu cây cho AdaBoost, hình dạng mạng cho MLP), và từ việc ghi lại trung thực cả những thí nghiệm không rõ ràng (tăng cường dữ liệu không cải thiện chắc chắn độ chính xác trên ảnh sạch với số seed đã chạy, nhưng cải thiện rõ độ bền với đúng loại biến đổi được huấn luyện).

Hướng tiếp theo. Chạy nhiều seed và nhiều phép chia cho mọi mô hình để có khoảng tin cậy thật; tìm kiếm tương tác siêu tham số (ví dụ tìm ngẫu nhiên hoặc Bayes) trong cùng ngân sách; tách riêng tác động của việc có thêm dữ liệu khỏi tác động của loại biến đổi trong tăng cường; đối

chiếu bản đồ kích hoạt với một phương pháp quy gán đã được kiểm chứng; hiệu chỉnh xác suất của AdaBoost; và kiểm tra bản demo trên dữ liệu viết tay thật ngoài MNIST.

10 Đóng góp của nhóm và công bố sử dụng AI

10.1 Phân công và đóng góp

Bảng dưới đây chỉ ghi những gì có thể kiểm chứng từ lịch sử Git của kho mã (`git shortlog`, các pull request #1–#4 và tệp `WORKFLOW.md`). Tỉ lệ đóng góp **50%–50%** do hai thành viên tự thống nhất và xác nhận (không suy ra từ số commit).

Thành viên	MSSV	Công việc có bằng chứng trong kho mã	Tỉ lệ
Lâm Nhật Tiến (GitHub: Tien-Lam28)	24280029	23 commit (13/10–06/11/2026) và PR #1–#4: dựng kho, README/WORKFLOW, tải và tiền xử lý dữ liệu, lớp cha `BaseClassifier`, độ đo, `Evaluator`, K-fold, Gaussian NB, bản đầu của MLP (backprop viết tay, early stopping), EDA, notebook trực quan. Theo `WORKFLOW.md` được phân công NB và MLP.	50%
Trần Trinh Nhân (GitHub: TranTrinhNhan1)	24280088	Khởi tạo kho; theo `WORKFLOW.md` được phân công mô hình tuyến tính, ensemble và phần mở rộng. Giai đoạn hoàn thiện (softmax regression, AdaBoost-SAMME, tái cấu trúc MLP/độ đo/tiền xử lý, bộ kiểm thử, pipeline thực nghiệm, xuất mô hình, website, báo cáo, slide) được thực hiện cùng trợ lý AI (xem dưới) và commit dưới danh nghĩa này.	50%

Bảng 12. Đóng góp theo bằng chứng Git. Họ tên và MSSV do nhóm cung cấp; tỉ lệ đóng góp (50%/50%) do nhóm xác nhận.

10.2 Công bố sử dụng công cụ AI

Theo yêu cầu của đề bài, nhóm công bố: phần hoàn thiện đồ án sau commit `f9a602b` – gồm `SoftmaxRegression`, `AdaBoostSAMME` và cây quyết định có trọng số, các sửa đổi cho `MLP`, độ đo, tiền xử lý và `Evaluator`, toàn bộ bộ kiểm thử, pipeline thực nghiệm và tích hợp Weights & Biases, quy trình xuất ONNX/Hugging Face, website, notebook tái lập, mã nguồn LaTeX của báo cáo này, slide, CNN, các thí nghiệm mở rộng và trình khám phá thực thi trên trình duyệt – được viết *với sự hỗ trợ của trợ lý lập trình AI Claude Code (Anthropic)*. Trợ lý đã chạy mã, kiểm thử và thí nghiệm trong môi trường của nhóm; mọi con số trong báo cáo được sinh từ các tệp kết quả do chương trình ghi ra. Các thành viên có trách nhiệm đọc, hiểu và giải thích được mã. Mức độ hỗ trợ của AI ở các giai đoạn trước đó (trước commit `f9a602b`) **[nhóm tự khai báo]**. Các tài liệu tham khảo được kiểm tra thủ công về tồn tại và thông tin thư mục.

Tài liệu tham khảo

[1] Lukas Biewald. Experiment tracking with Weights and Biases, 2020. Software available from https://www.wandb.com/.

[2] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

[3] Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Wadsworth, 1984.

[4] Richard O. Duda, Peter E. Hart, and David G. Stork. *Pattern Classification*. Wiley, 2 edition, 2001.

[5] Yoav Freund and Robert E. Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139, 1997.

[6] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034, 2015.

[7] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[8] Lutz Prechelt. Early stopping – but when? In *Neural Networks: Tricks of the Trade*, volume 1524 of *Lecture Notes in Computer Science*, pages 55–69. Springer, 1998.

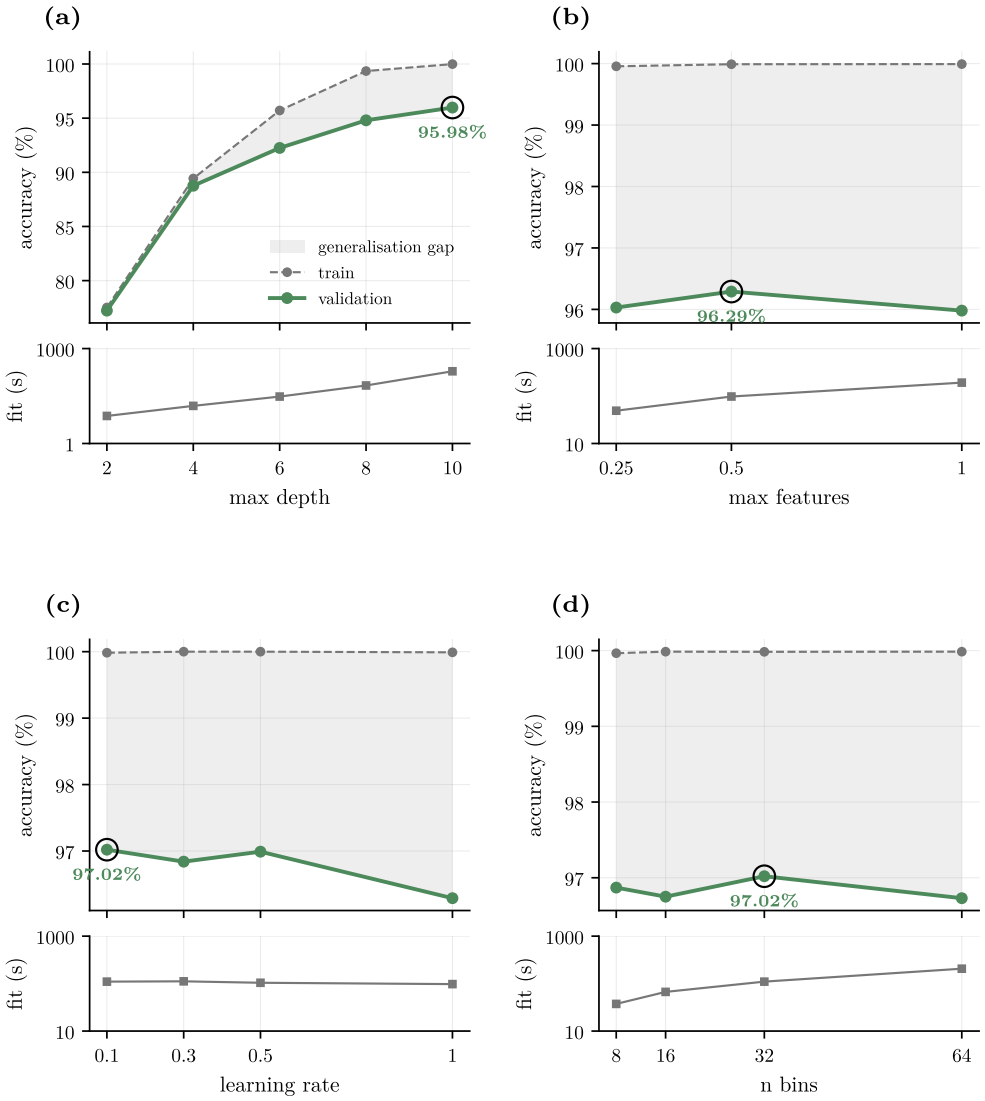
[9] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

[10] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014.

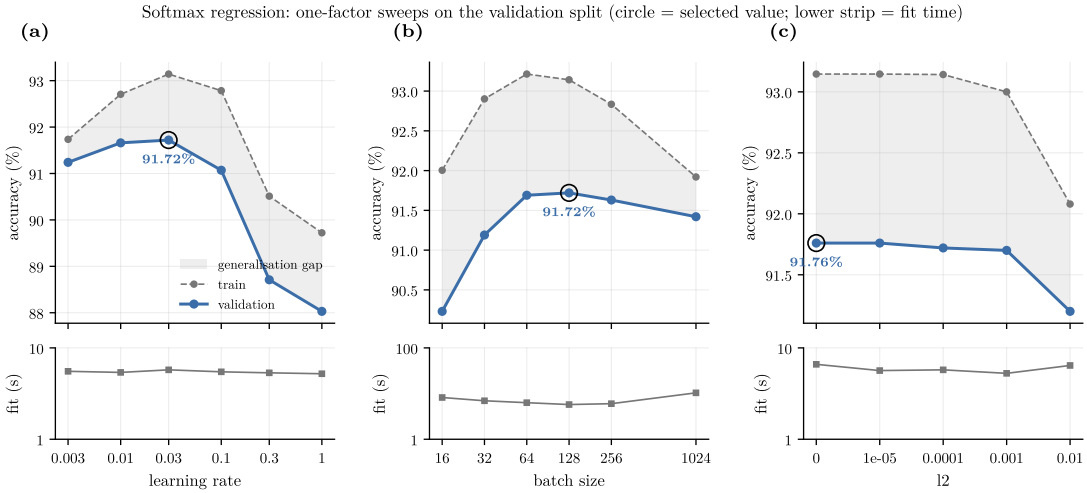
[11] Ilya Sutskever, James Martens, George Dahl, and Geoffrey Hinton. On the importance of initialization and momentum in deep learning. In *Proceedings of the 30th International Conference on Machine Learning (ICML)*, volume 28 of *PMLR*, pages 1139–1147, 2013.

[12] Ji Zhu, Hui Zou, Saharon Rosset, and Trevor Hastie. Multi-class AdaBoost. *Statistics and Its Interface*, 2(3):349–360, 2009.

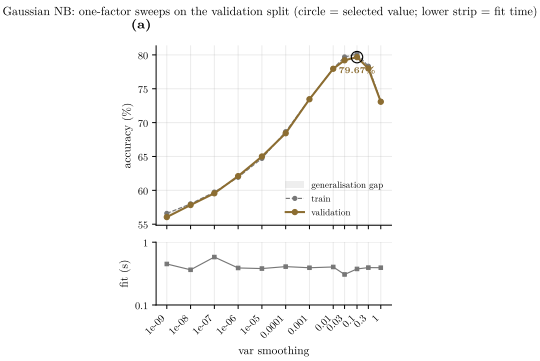
A Các lượt quét siêu tham số còn lại



Hình 13. AdaBoost-SAMME: quét từng siêu tham số trên validation (số vòng boosting chọn riêng từ đường cong, hình 5c).



Hình 14. Hồi quy softmax: quét từng siêu tham số trên validation.



Hình 15. Gaussian Naïve Bayes: quét `var_smoothing` trên validation.

B Tái lập kết quả

Môi trường dùng `uv` (Python $\geq$ 3.11); mã, kiểm thử và dòng lệnh nằm trong kho GitHub của đồ án.

```
uv sync --extra track --extra export
uv run python -m src prepare          # tải MNIST, chia train/val, chuẩn hóa
uv run python -m src tune --model all # quét siêu tham số, ghi results/tuning
uv run python -m src train --model all
uv run python -m src baselines --model all   # scikit-learn
uv run python -m src augment          # thí nghiệm tăng cường dữ liệu
uv run python -m src report           # bảng, hình, các đoạn LaTeX
uv run python -m src export           # ONNX + kiểm tra số học
uv run python -m src upload           # đẩy lên Hugging Face (cần token)
uv run pytest
```

Báo cáo biên dịch bằng `pdflatex` + `bibtex` trong thư mục `report/`; các bảng và số liệu nằm trong `results/tables` và `report/generated` do bước `report` sinh ra.